

LAPORAN EKSPLOITASI

Proof of Concept — FLAG 1

Digital Library System
CTF Web Exploitation
2026-05-03

Severity: Medium

Type: Information Disclosure

Status: Confirmed

1. Objektif Tantangan

Menemukan dan mengekstrak nilai **ISBN buku tertua** dari seluruh data buku yang ada di dalam sistem dengan memanfaatkan celah keamanan pada fitur pencarian aplikasi untuk mendapatkan akses ke seluruh basis data buku.

2. Informasi Kerentanan

Jenis Kerentanan	Information Disclosure / Broken Access Control
Tingkat Keparahan	Medium
Target Endpoint	/backend/api.php?action=search&q=
HTTP Method	GET
Status Code	200 OK

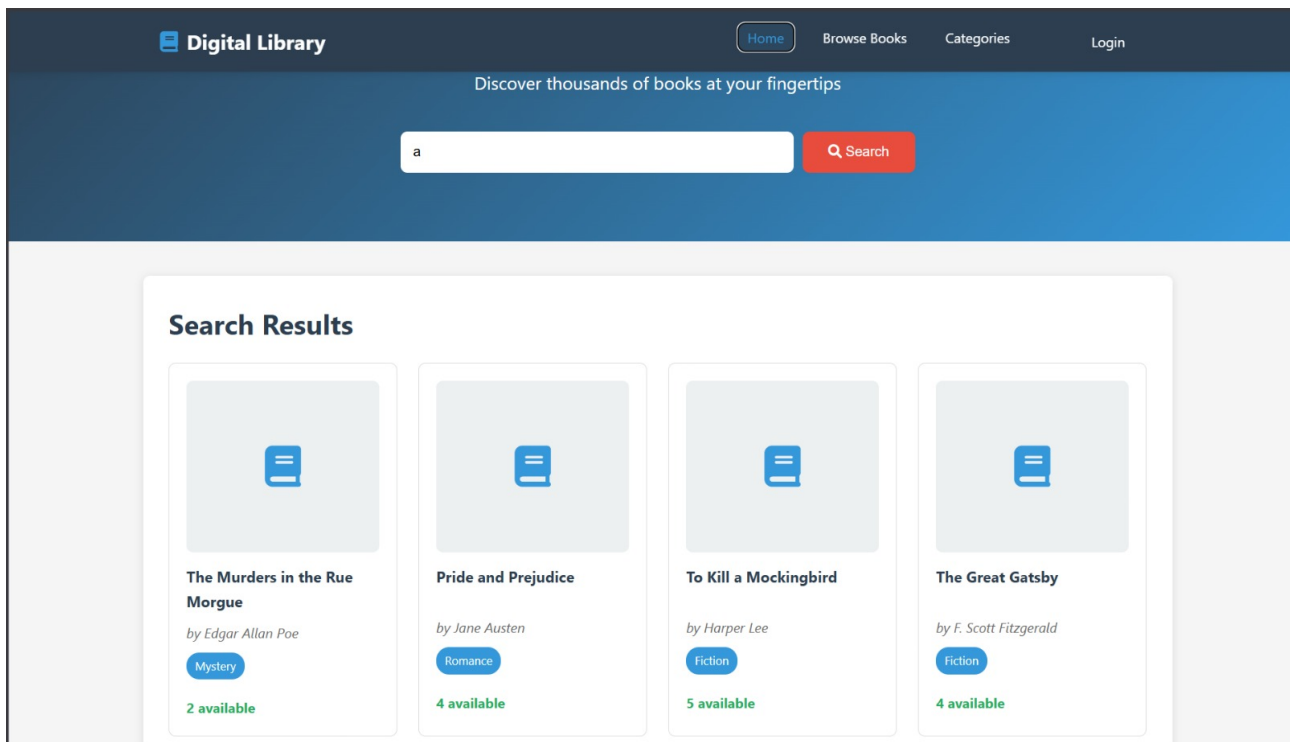
3. Deskripsi Bug

Terdapat kelemahan pada fitur pencarian aplikasi di mana sisi server (backend) **gagal memvalidasi parameter pencarian** (query string). Dengan mengosongkan parameter **search** pada URL, fungsi filter pada database gagal beroperasi sebagaimana mestinya. Akibatnya, server merespons dengan memberikan seluruh kumpulan data buku secara utuh. Celah ini memungkinkan penyerang untuk menarik seluruh informasi katalog dan menyelesaikan tantangan dengan sangat mudah melalui data JSON yang bocor.

4. Langkah-Langkah Reproduksi (Proof of Concept)

Langkah 1: Observasi Permintaan Normal

Akses halaman pencarian buku pada aplikasi dan lakukan pencarian dengan memasukkan kata kunci acak (misalnya: **a**). Tampilan antarmuka web saat fitur pencarian dieksekusi:

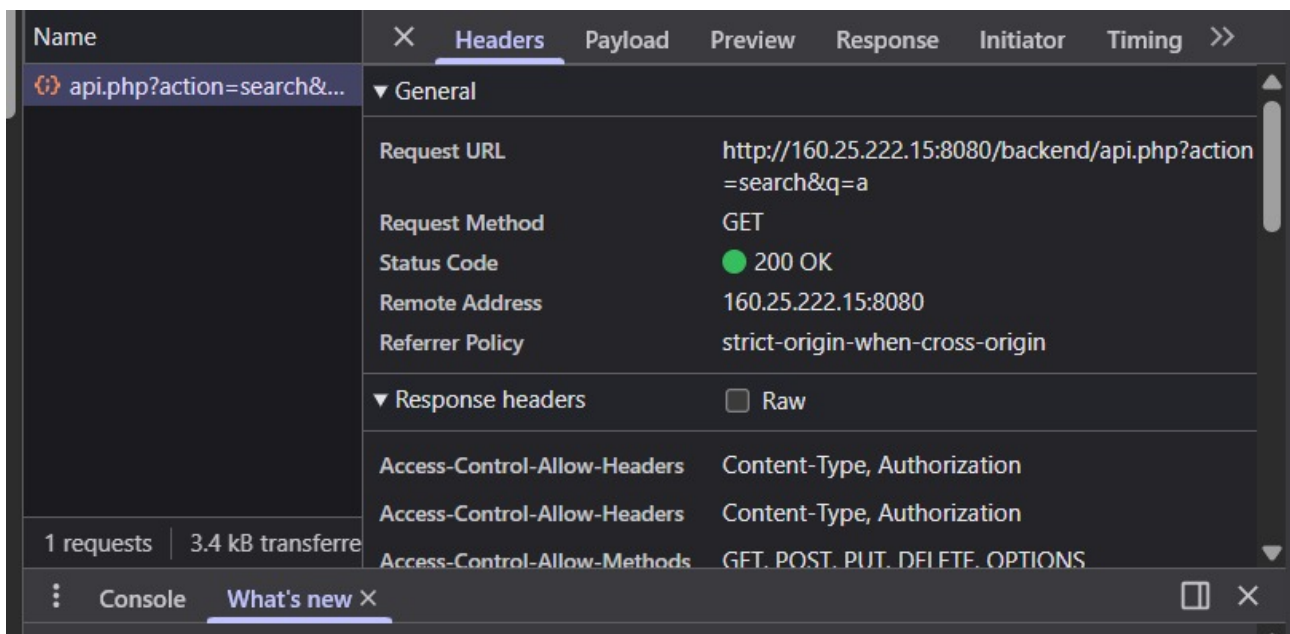


Gambar 1 — Antarmuka web Digital Library saat melakukan pencarian dengan input kata kunci.

Langkah 2: Inspeksi Endpoint via DevTools

Buka **Developer Tools** pada browser (F12), arahkan ke tab **Network**. Analisis request yang berjalan saat fitur pencarian dieksekusi. Ditemukan bahwa aplikasi melakukan permintaan GET ke:

GET <http://160.25.222.15:8080/backend/api.php?action=search&q=a>



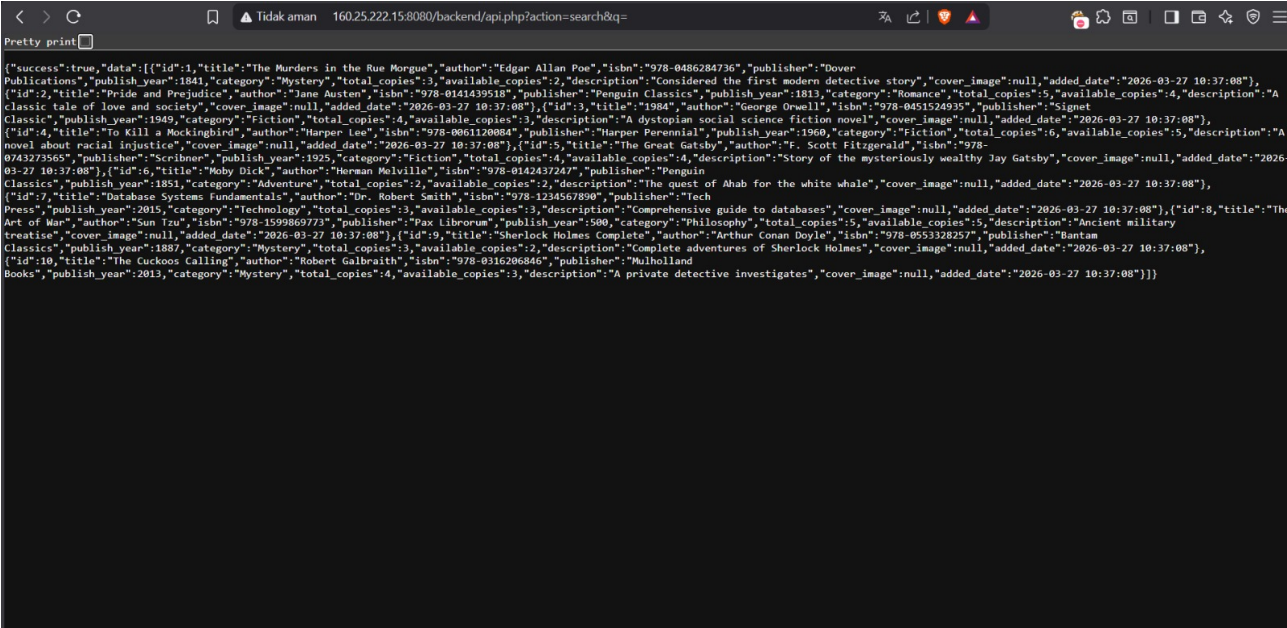
Gambar 2 — Tab Network di DevTools menunjukkan URL API, HTTP Method, dan Response Headers.

Langkah 3: Manipulasi Parameter (Eksplorasi)

Lakukan manipulasi pada URL tersebut dengan **menghapus** nilai dari parameter **q** sehingga string menjadi kosong:

GET <http://160.25.222.15:8080/backend/api.php?action=search&q=>

Akses atau kirim ulang permintaan tersebut secara langsung melalui browser. Server merespons dengan mengembalikan **seluruh data buku** dalam format JSON tanpa pembatasan apapun:



Gambar 3 — Respons server berisi raw JSON dump seluruh katalog buku dalam database.

Langkah 4: Penyelesaian Tantangan

Setelah seluruh data buku berhasil ditarik dalam format JSON, data dapat disortir berdasarkan **publish_year** untuk menemukan buku tertua. Hasil yang diperoleh:

Field	Value
Judul Buku Tertua	Pride and Prejudice
Penulis	Jane Austen
Penerbit	Penguin Classics
Tahun Terbit	1813
ISBN (FLAG)	978-0141439518

5. Dampak Kerentanan

- **Kebocoran Data (Data Exposure):** Pengguna yang tidak berwenang dapat melakukan scraping terhadap seluruh katalog database tanpa terdeteksi atau terblokir.
- **Degradasi Kinerja Server:** Kueri tanpa batasan parameter berpotensi membebani memori server dan database jika jumlah data bertambah besar, yang bisa berujung pada kondisi *Denial of Service* (DoS).

6. Mitigasi dan Rekomendasi

1. **Validasi Input Secara Ketat:** Konfigurasi server agar menolak kueri (mengembalikan error 400 Bad Request) atau mengembalikan array kosong apabila parameter pencarian tidak diisi.
2. **Implementasi Pagination:** Wajibkan penggunaan *limit* dan *offset* pada API pencarian (misalnya membatasi respons maksimal 20 data per halaman) untuk mencegah pengunduhan seluruh database dalam satu kali permintaan.